

프로세스에 입각한 바이브코딩의 효과 분석

본 보고서는 AI 보조 개발을 단순한 빠른 구현 수단으로 사용했을 때와 달리, 요구사항 분석, 설계 결정, 구현, 검증, 회고를 연결한 소프트웨어공학 프로세스 안에서 바이브코딩을 수행했을 때 어떤 효과가 나타나는지 분석한다.

SW 산출물

한국 지역 불균형 대시보드

저장소

github.com/Bias92/korea-regional-imbalance

배포 데모

bias92.github.io/korea-regional-imbalance

앱 형태

GitHub Pages 기반 정적 웹앱 + PWA 설치성

AI 도구

Claude, Codex

기준일

2026-06-02

1. 프로젝트 개요

프로젝트 주제는 수도권 집중, 인구감소지역, 폐교 지표를 한 화면에서 비교하는 정적 웹 대시보드 개발이다. 통계 자료는 표 형태로 제공되는 경우가 많아 일반 사용자가 지역 간 차이를 직관적으로 이해하기 어렵다. 따라서 본 프로젝트는 카드, 차트, 지도, 위험지수, 지역 비교, 정책 대응 시뮬레이터를 통해 지역 불균형을 탐색 가능한 화면으로 구성했다. 최종 산출물은 네이티브 앱이 아니라 브라우저에서 실행되는 정적 웹앱이지만, manifest와 service worker를 추가해 지원 브라우저에서는 앱처럼 설치할 수 있도록 했다.

이 프로젝트의 목적은 단지 동작하는 웹앱을 만드는 것이 아니다. 과제의 핵심은 바이브코딩 과정에 소프트웨어공학 프로세스를 적용했을 때의 효과를 분석하는 것이다. 따라서 기능 구현과 동시에 요구사항 명세, 설계 결정 기록, 테스트 계획, AI 작업 로그, lessons learned를 GitHub 저장소에 지속적으로 누적했다.

2. 적용 방법론

본 프로젝트는 단일 방법론을 기계적으로 적용하지 않고, 단기간 데이터 시각화 과제의 특성에 맞춰 위험 기반 반복·점진 개발을 중심으로 운영했다. 여기에 V-Model식 추적성과 ADR 기반 의사결정 기록을 결합했다.

반복·점진 개발

Phase 1 MVP를 먼저 확보한 뒤 지도, 위험지수, 폐교 분석, 정책 시뮬레이션, PWA 설치성, 보고서 기능을 순차 추가했다.

위험 기반 스코프 관리

시군구 지도, 실시간 API, 폐교 위치 레이어처럼 일정 위험이 큰 기능은 후순위로 분리했다.

V-Model식 추적성

요구사항 ID를 설계 결정, 구현 파일, 테스트 항목에 연결해 변경 이유와 검증 기준을 남겼다.

ADR과 회고

주요 설계 판단은 `docs/design.md`, 과정의 교훈은 `docs/lessons-learned.md`에 기록했다.

요구사항

사용자, 평가자, 리뷰어 관점 정의

설계

정적 JSON, Vanilla JS, 지도 범위 결정

구현

대시보드 기능을 Phase별 누적

검증

수동 체크리스트와 자동 테스트 실행

회고

AI 로그와 lessons learned 갱신

3. 요구사항 분석과 범위 관리

주요 사용자는 지역 불균형 문제를 빠르게 이해하려는 일반 사용자, 과제 평가자, 프로젝트 진행 방식과 AI 활용을 확인하려는 리뷰어로 정의했다. 일반 사용자는 수치를 한눈에 파악하고 지역을 비교해야 하며, 평가자는 개발 과정이 요구사항 분석부터 품질 관리까지 체계적으로 이어졌는지 확인해야 한다.

구분	핵심 요구사항	프로세스상 의미
기능 요구사항	통계 카드, 차트, 지도, 위험지수, 지역 비교, 정책 시뮬레이터, 폐교 분석, PWA 설치성, 캡처 모드	단순 표시 화면에서 탐색형 분석 도구로 확장
비기능 요구사항	정적 호스팅, 반응형 레이아웃, 재현성, 유지보수성, 접근성	제출 환경에서 별도 서버 없이 재현 가능한 산출물 확보
범위 제외	실시간 API, 사용자 계정, 시군구 정밀 지도, 폐교 위치 좌표 레이어	일정과 데이터 리스크를 통제하기 위한 의도적 스코프 관리

요구사항은 `docs/requirements.md`에 FR-001부터 FR-026까지 ID로 관리했다. 새 기능을 추가할 때는 요구사항과 설계 결정, 테스트 계획을 함께 갱신하여 AI가 만든 코드가 문서 밖에서 떠돌지 않도록 했다.

4. 구현 산출물

구현 결과물은 HTML, CSS, Vanilla JavaScript 기반의 정적 웹앱이다. Chart.js로 막대차트를, Leaflet과 저장소 내 TopoJSON 파일로 시도 단위 지도를 렌더링한다. 별도 빌드 없이 GitHub Pages 또는 로컬 정적 서버에서 실행할 수 있도록 구성했다. 또한 `manifest.webmanifest`, 앱 아이콘, `service-worker.js`를 추가하여 PWA 설치 조건을 갖추었다.

한국 지역 불균형 대시보드

수도권 집중·인구감소·폐교 지표를 한 화면에서 비교합니다.

수도권 인구 비중

50.8 %

서울·인천·경기 기준

출처: 통계청 인구주택총조사(등록센서스) (2024)

수도권 GRDP 비중

52.3 %

경제활동 집중도 비교

출처: 통계청 지역소득(잠정) (2023)

인구감소지역

89 곳

시군구 단위 지정 지역

출처: 행정안전부 인구감소지역 지정 고시 (2021 지정·2026 유지)

폐교 수

3,922 곳

전국 누적 폐교 수

출처: 교육부 지방교육재정알리미 폐교 현황 (누계)

DECISION BRIEF

핵심 진단

균형

인구감소 중심

소멸위험 중심

폐교 중심

인구 감소 40%, 인구감소지역 35%, 폐교 수 25%를 반영합니다.

위험지수 산식

위험지수 = 인구 감소 40% + 인구감소지역 35% + 폐교 수 25%

균형 기준입니다. 각 항목은 17개 시도 내 최댓값 대비 0~100으로 정규화한 뒤 가중합합니다.

인구 감소

40%

인구감소지역

35%

폐교 수

25%

균형 기준

전남 94점

매우 높은 등급입니다. 인구감소지역 16곳, 폐교 823 곳이 함께 잡힙니다.

감소 신호

부산 -3.2%

17개 시도 중 인구 증감률이 가장 낮습니다. 시도에서 위험지수와 함께 보면 감소 폭과 지역 기반 지표를 같이 판단할 수 있습니다.

격차 요약

1.4%p 차이

수도권 평균 증감률은 +1.3%, 비수도권 평균은 -0.1%입니다. 폐교 수 최대 지역은 전남입니다.

REGION COMPARE

지역 비교

지역 A

전남

지역 B

경기

위험지수 차이

전남 +84점

전남은 경기보다 위험지수가 높습니다. 전남 94점, 경기 10점입니다.

인구 흐름

-6.2%p

전남의 인구 증감률은 -2.7%, 경기는 +3.5%입니다.

지역 기반 지표

+14곳 / +638곳

인구감소지역 차이는 +14곳, 폐교 수 차이는 +638 곳입니다.

ACTION BRIEF

대응 전략

우선 대응

전남 94점

균형 기준 최상위 지역입니다. 2순위 경북보다 4점 높고, 가장 큰 기여 요소는 인구감소지역입니다.

핵심 조치

생활권 단위 대응

전남은 인구감소지역 16곳이 잡힙니다. 개별 시군구보다 생활권 단위로 의료·교통·돌봄 접근성을 묶어 봐야 합니다.

검증 과제

시도별 공식값 교체

요약 지표는 공식 통계로 교체됐지만 시도·위험지수 입력값은 아직 임시 데이터입니다. 다음 반박에서는 시도별 공식 인구·폐교 데이터를 먼저 확정해야 합니다.

POLICY LAB

대응 시뮬레이터

대상 지역

전남 · 94점

청년·일자리

55%

예상 위험 변화

94점에서 78점

16점 완충되는 개선이 나타납니다.

집중 패키지

청년·일자리 55%

최저 일자리, 저소득 지역, 지역 사업

전역 리스크

인구감소지역

균형 기준으로 조정된 미래도

그림 1. 2026-06-02 기준 데스크톱 대시보드 캡처

PHASE 3 IN PROGRESS

한국 지역 불균형 대시보드

수도권 집중·인구감소·폐교 지표를 한 화면에서 비교합니다.

캡처 모드

혼합 데이터

수도권 인구 비중

50.8 %

서울·인천·경기 기준

출처: 통계청 인구주택총조사(등록센서스) (2024)

수도권 GRDP 비중

52.3 %

경제활동 집중도 비교

출처: 통계청 지역소득(잠정) (2023)

인구감소지역

89 곳

시군구 단위 지정 지역

출처: 행정안전부 인구감소지역 지정 고시 (2021 지정·2026 유지)

폐교 수

3,922 곳

전국 누적 폐교 수

출처: 교육부 지방교육재정알리미 폐교 현황 (누계)

DECISION BRIEF

핵심 진단

균형

인구감소 중심

소멸위험 중심

폐교 중심

인구 감소 40%, 인구감소지역 35%, 폐교 수 25%를 반영합니다.

위험지수 산식

그림 2. 2026-06-02 기준 모바일 대시보드 캡처

5. 검증과 품질 관리

품질 관리는 수동 확인과 자동 검증을 병행했다. 정적 웹앱은 빌드 단계가 없기 때문에 DOM id 변경, JSON 구조 오류, 지도 경계 매핑 오류가 런타임에서야 드러날 수 있다. 이를 막기 위해 Node.js 기반 검증 스크립트를 누적했다.

검증 항목	도구	확인 내용
문법 검사	<code>`node --check src/app.js`</code>	JavaScript 구문 오류 확인
데이터 검증	<code>`tests/verify-data.mjs`</code>	17개 시도, 필수 요약 카드, 지도 경계 매핑 확인
UI 계약 검증	<code>`tests/verify-ui-contract.mjs`</code>	핵심 DOM id, 버튼, 렌더링 함수, 요구사항 추적성 확인
원격 검증	GitHub Actions	push마다 <code>`npm test`</code> 실행 후 Pages 배포

2026-06-02 기준 ``npm test`` 와 ``git diff --check`` 를 모두 통과했다. 이 검증 기록은 ``docs/test-plan.md`` 와 ``docs/ai-log/`` 에 함께 남겼다.

6. 바이브코딩의 효과 분석

프로세스 없는 바이브코딩은 빠르게 화면을 만들어 주지만, 기능이 늘어날수록 왜 만들었는지, 어떤 요구사항을 만족하는지, 무엇을 검증했는지 추적하기 어렵다. 반대로 프로세스 안에서 AI를 활용하자 다음 효과가 나타났다.

속도와 안정성의 균형 AI가 구현 속도를 높였고, 요구사항 ID와 테스트 계획이 기능 추가의 기준선을 제공했다.	스코프 크리프 억제 AI가 제안한 큰 기능을 모두 수용하지 않고 Phase와 리스크 기준으로 선별했다.
설명 가능한 산출물 설계 결정과 lessons learned를 남겨 결과물뿐 아니라 판단 과정도 제출 가능해졌다.	회귀 방지 데이터 검증과 UI 계약 검증을 추가해 기능 확장 중 핵심 화면이 깨지는 위험을 줄였다.

특히 Claude는 요구사항 정리와 보고서 구조 논의에, Codex는 구현, 테스트, 커밋 정리에 활용했다. 두 AI의 역할을 분리하고 결과를 교차 검토하자 단일 AI의 낙관적 일정 추정이나 과도한 기능 제안을 보정할 수 있었다.

7. 프로세스 적용의 교훈

교훈 1. MVP는 탈출 경로다

지도 구현이 지연되더라도 카드와 차트만으로 제출 가능한 Phase 1 결과물을 먼저 확보했다.

교훈 2. 데이터 신뢰도는 기능의 일부다

공식값과 임시값을 구분해 표시함으로써 분석 결과의 한계를 화면과 보고서에 함께 드러냈다.

교훈 3. 문서도 품질이다

README, 요구사항, 설계, 테스트, 회고 문서를 연결해 채점자가 흐름을 따라갈 수 있게 했다.

교훈 4. 캡처도 형상 관리 대상이다

보고서에 들어갈 화면 캡처를 저장소에 커밋해 코드와 같은 시점의 증빙으로 남겼다.

교훈 5. 데모는 조작성이 필요하다

정책 대응 시뮬레이터를 추가해 사용자가 지역과 투입 강도를 바꾸며 결과 변화를 확인하게 했다.

8. 한계와 향후 개선

본 프로젝트는 요약 지표 일부를 공식 통계로 교체했지만, 시도별 분석값 중 일부는 아직 화면 검증용 임시 데이터다. 따라서 위험지수 순위는 정책 판단의 절대 기준이 아니라, 탐색과 비교를 돕는 데모 지표로 해석해야 한다.

향후에는 시도별 인구 증감률과 폐교 수를 모두 공식 기준연도로 교체하고, 위험지수 가중치와 등급 구간을 재검토해야 한다. 또한 최종 제출 전 GitHub Pages 배포 결과와 PDF 렌더링 상태를 한 번 더 검증해야 한다.

9. 결론

프로세스에 입각한 바이브코딩은 빠른 구현과 체계적 품질 관리를 함께 가능하게 했다. AI는 코드와 문서 초안을 빠르게 생성했지만, 요구사항 분석, 스코프 결정, 설계 기록, 검증 기준, 회고가 없었다면 결과물은 단순한 데이터 표시 화면에 머물렀을 것이다.

본 프로젝트에서는 위험 기반 반복·점진 개발을 통해 MVP를 먼저 확보하고, 지도, 위험지수, 지역 비교, 정책 대응 시뮬레이터, 폐교 분석, 데이터 신뢰도, PWA 설치성, 보고서 캡처 기능을 단계적으로 추가했다. 그 과정에서 요구사항 ID, 설계 결정, 자동 테스트, AI 작업 로그, lessons learned를 GitHub에 누적했다. 따라서 바이브코딩의 효과는 단순히 개발 속도 향상에 그치지 않고, 프로세스를 통해 재현 가능하고 설명 가능한 소프트웨어 산출물로 전환되는 데 있음을 확인했다.